Progressive Cramming: Reliable Token Compression and What It Reveals

Anonymous Authors[1]

Abstract

Token cramming compresses sequences into learned embeddings with near-perfect reconstruction, but prior work used fixed token budgets and 99% accuracy thresholds, obscuring whether residual errors reflect optimization failures or fundamental limits. We introduce progressive cramming, which grows the target prefix token-by-token and stops only when reconstruction is no longer achievable within a fixed optimization budget. Using progressive trajectories, we find that optimization paths occupy surprisingly low-dimensional structure in the embedding space. Attention analysis shows that compression embeddings often become attention sinks in specific intermediate layers, which correlates with both optimization difficulty and downstream degradation. On likelihood-based multiple-choice evaluation, prepending a crammed embedding drops accuracy to close to random guessing, even with the original prefix in context. These results suggest that perfect reconstruction can arise from brittle steering and attention hijacking, rather than a transferable semantic representation. Our results position progressive cramming as a tool for studying compression limits, while showing that perfect reconstruction is insufficient for meaningful compression.

1. Introduction

How much information can a single embedding encode? Recent work on "cramming" (Kuratov et al., 2025) probed this question by optimizing embeddings to reconstruct token sequences through autoregressive decoding. The results were striking: a single input embedding can encode up to 1568 tokens with near-perfect reconstruction, suggesting substantial latent capacity in transformer representations.

[1]Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

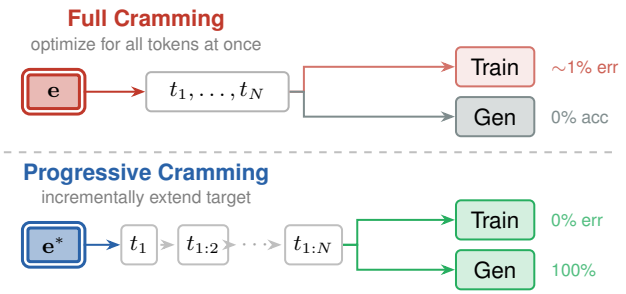

Figure 1. Full cramming achieves ∼1% training error with teacher forcing, but errors cascade to 0% generation accuracy. Progressive cramming establishes a precise boundary where compression either succeeds completely or fails clearly.

But *how* do transformers achieve this? The original work focused on capacity limits – how many tokens fit – rather than the mechanism enabling reconstruction. This leaves open a fundamental question: does cramming discover dense semantic representations that the model can interpret, or does it exploit some other property of transformer computation? Understanding this mechanism matters beyond cramming: it can shed light on how transformers use information stored in embedding space.

We first address methodological limitations that complicate controlled study. Prior work used fixed token budgets and 99% accuracy thresholds with teacher forcing. But the last 1% of errors may lead to catastrophic failures in autoregressive generation without teacher forcing. We introduce *progressive cramming*: sequentially adding tokens until perfect reconstruction, with two stabilizing techniques – activation alignment and low-dimensional projection. This achieves reliable 100% accuracy and enables precise measurement of where compression fails. Figure 1 shows comparison of full and progressive cramming frameworks.

With these tools, we investigate the cramming mechanism. Analysis of optimization trajectories shows they occupy surprisingly low-dimensional manifolds: 30-100 PCA components explain 99% of variance in 2048-4096 dimensional embedding spaces. More critically, attention pattern analysis indicates that reconstruction is associated with attention hijacking: compression embeddings capture 20-60% of attention mass in intermediate layers. We use the term *atten-*

tion hijacking to emphasize that this attention concentration is associated with a breakdown of downstream language modeling capabilities, rather than benign attention focusing. This concentration is invariant to sequence length – the same layers are dominated whether cramming 32 or 512 tokens – consistent with compression engaging relatively fixed circuits rather than scaling information density.

This mechanism has consequences. If cramming worked through semantic encoding, compressed prefixes should preserve downstream capabilities. Evaluation on HellaSwag and ARC shows the opposite: accuracy drops toward random guessing, even when the original prefix remains in context. Together with the attention results, this suggests that cramming can achieve reconstruction via brittle steering (e.g., attention capture) rather than by encoding information the model can flexibly use.

These findings reframe what cramming reveals about transformers. The high capacity demonstrated by prior work reflects not dense semantic encoding but the ease of hijacking attention mechanisms. This distinction matters for future work on learned compression: methods that override computation can achieve perfect reconstruction while encoding nothing transferable.

Our main contributions are as follows:

- We introduce progressive cramming, which grows the target prefix token-by-token and targets 100% reconstruction, enabling a sharp success/failure boundary beyond fixed-budget, 99% protocols.
- We show that progressive optimization trajectories $\{\mathbf{e}^{(k)}\}$ are low-dimensional: across model families, 30–100 PCA components explain 99% of per-sample trajectory variance in 2048–4096 dimensional embedding spaces.
- We quantify attention hijacking during cramming via attention mass to position 0 and find that compression embeddings can capture substantial mass in specific intermediate layers (often 20–60%), with strong model-family dependence.
- We demonstrate that optimizing for reconstruction does not imply preserved downstream utility: on likelihood-based multiple-choice evaluation (HellaSwag, ARC-Easy), prepending a crammed compression embedding reduces accuracy to close to random guessing across model families, even when the original prefix remains in context.

2. Related Work

2.1. Token Cramming

Our work is built upon token cramming (Kuratov et al., 2025), where per-sample optimization compresses up to 1,568 tokens into a single Llama-3.1-8B input embedding ($\sim$1500$\times$ compression), far beyond typical encoder-based lossless ratios ($\leq 10\times$). Crucially, they show that capacity is better characterized by cross-entropy reduction than raw token count, suggesting the model leverages internal knowledge to complement stored information.

Extending this line of inquiry, Mezentsev & Oseledets (2025) show that frozen LLMs can generate hundreds of accurate tokens non-autoregressively from just two learned "proto-token" embeddings, and that valid compression representations form connected local regions in embedding space.

2.2. Context Compression and Distillation

A parallel line of research addresses context compression for the purpose of reducing computational costs during inference. Importantly, these approaches differ fundamentally from token cramming in their objective: they focus on *semantic compression* that preserves task-relevant information for downstream use, rather than *exact reconstruction* of the original token sequence.

Zhang et al. (2025) propose Activation Beacon, which compresses per-layer keys and values into special beacon tokens. This activation-level compression enables flexible compression ratios while preserving short-context capabilities, and supports progressive chunk-wise processing up to 400K tokens.

Chevalier et al. (2023) propose AutoCompressors, which learn unsupervised summary vectors that compress long contexts. These vectors act as soft prompts, letting the model condition on compressed representations of earlier segments as it processes a document sequentially.

The In-Context Autoencoder (ICAE) (Ge et al., 2024) uses a LoRA-adapted encoder to compress contexts into memory slots that the target LLM conditions on, achieving $4\times$ compression while retaining prompt-following.

These methods target semantic compression for downstream tasks, whereas we study near-lossless reconstruction as a cleaner probe of embedding-based information storage.

2.3. Soft Prompts and Prefix Tuning

The optimization of continuous embeddings as model inputs originated in parameter-efficient fine-tuning research. Li & Liang (2021) introduce prefix-tuning, which prepends learnable continuous vectors to the input that subsequent

tokens can attend to as "virtual tokens." By optimizing only 0.1% of parameters while keeping the language model frozen, prefix-tuning achieves comparable performance to full fine-tuning on generation tasks while enabling efficient multi-task deployment.

Lester et al. (2021) propose prompt tuning, a simplified approach that learns soft prompts only at the input layer rather than at every transformer layer. They demonstrate that prompt tuning becomes increasingly competitive with full model tuning as model scale increases, with billion-parameter models closing the gap entirely. This finding suggests that larger models develop embedding spaces with sufficient structure to support rich task-specific conditioning through input manipulation alone.

These methods establish that frozen language models can be effectively steered through learned embeddings without modifying model parameters. Token cramming can be viewed as an extreme case of this paradigm: rather than encoding task-specific instructions, the learned embedding must encode the complete information content of an arbitrary token sequence. Our progressive cramming methodology bridges these perspectives by studying how compression tokens interact with the model's attention and activation patterns as the encoding task becomes increasingly demanding.

3. Background: Token Cramming

3.1. Problem Formulation

In the cramming task, a single embedding is optimized to encode as many tokens as possible through autoregressive reconstruction. The language model weights remain frozen throughout optimization, and each sequence receives a newly initialized compression embedding.

Let $\mathcal{M}$ be an autoregressive language model with vocabulary $\mathcal{V}$ and embedding dimension d. Given a target sequence $\mathbf{x} = (x_1, \ldots, x_n)$ where $x_i \in \mathcal{V}$, the cramming objective finds an embedding $\mathbf{e} \in \mathbb{R}^d$ such that:

$$\mathbf{e}^* = \arg\min_{\mathbf{e}} \mathcal{L}_{\text{cram}}(\mathbf{e}; \mathbf{x}, \mathcal{M}) \tag{1}$$

where the cramming loss is the cross-entropy over the target sequence:

$$\mathcal{L}_{\text{cram}}(\mathbf{e}; \mathbf{x}, \mathcal{M}) = -\sum_{i=1}^{n} \log p_{\mathcal{M}}(x_i \mid \mathbf{e}, x_1, \ldots, x_{i-1}) \tag{2}$$

3.2. Limitations of Prior Work

Prior work on prompt compression and reconstruction typically evaluates success using a fixed accuracy threshold (e.g., 99% token-level reconstruction under teacher forcing), often in conjunction with a fixed token budget. Our results indicate that this evaluation protocol substantially underestimates critical failure modes that emerge during autoregressive generation.

The "last 1%" problem. While a 99% reconstruction accuracy suggests near-perfect training performance, the remaining errors are not uniformly distributed nor benign. As shown in Table 18, reconstruction mismatches are almost entirely concentrated at the earliest token positions, specifically at indices 0 and 1. Even when mean teacher-forcing convergence exceeds 97–99%, the probability of an incorrect prediction at the first or second token during inference approaches 100% across most model sizes and training variants.

Early-token errors induce autoregressive collapse. Errors at the earliest positions have a disproportionate impact under greedy decoding. Once the autoregressive process deviates at the first or second token, all subsequent tokens are generated conditioned on an incorrect prefix, leading to rapid and irreversible divergence. Consistently across all evaluated models, we observe that mean greedy convergence drops below 2% whenever compression training fails to achieve perfect reconstruction. In practice, a single early mismatch nullifies the utility of the remaining correctly reconstructed tokens, despite high aggregate training accuracy.

Fixed-budget evaluation obscures brittleness. Finally, evaluation under a fixed token budget further masks this brittleness. Methods with similar final convergence values may exhibit qualitatively different failure modes, depending on whether reconstruction errors occur early or late in the sequence. As demonstrated in Table 18, high average token-level accuracy does not guarantee usable autoregressive behavior. Instead, perfect reconstruction of the earliest tokens emerges as a necessary condition for reliable decoding from compressed representations.

4. Method: Progressive Cramming

4.1. Progressive Token Addition

Rather than optimizing a compression embedding for a fixed-length target, we progressively extend the target prefix and warm-start optimization:

1. Initialize with $\mathbf{x}^{(1)} = (x_1)$ and optimize $\mathbf{e}^{(1)} \in \mathbb{R}^d$ until perfect reconstruction.
2. Extend to $\mathbf{x}^{(k)} = (x_1, \ldots, x_k)$, initialize $\mathbf{e}^{(k)} \leftarrow \mathbf{e}^{(k-1)}$, and continue optimizing for the longer prefix.
3. Repeat until perfect reconstruction is no longer achievable within the optimization budget.

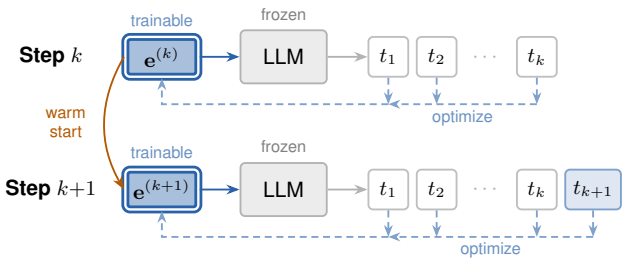

Figure 2. Progressive cramming adds target tokens sequentially. At stage k, we optimize the compression representation to achieve 100% reconstruction of the prefix $(x_1, \ldots, x_k)$, then extend to $k{+}1$ and warm-start from the previous solution.

This process is visualized on Figure 2.

Progressive cramming produces a sequence of embeddings $\{\mathbf{e}^{(k)}\}_{k=1}^{n}$, where $\mathbf{e}^{(k)}$ is the optimized compression embedding for the length-k prefix. This lets us quantify how far the optimizer moves in embedding space as new tokens are added. We define the (stage-wise) trajectory length as the sum of Euclidean distances between consecutive stages:

$$L_{\text{traj}} = \sum_{k=1}^{n-1} \left\| \mathbf{e}^{(k+1)} - \mathbf{e}^{(k)} \right\|_2 \tag{3}$$

4.2. Activation Alignment

To stabilize optimization, we regularize hidden states toward those of the uncompressed sequence. Let $\mathbf{h}_l^{\text{cram}}(i)$ denote the hidden state at layer l and token position i when the model is conditioned on the compression embedding, and let $\mathbf{h}_l^{\text{clean}}(i)$ be the corresponding state for the same token position without compression. We align activations using cosine distance:

$$\mathcal{L}_{\text{align}} = \frac{1}{n} \sum_{l=1}^{K} \sum_{i=1}^{n} \left(1 - \frac{\mathbf{h}_l^{\text{cram}}(i) \cdot \mathbf{h}_l^{\text{clean}}(i)}{\|\mathbf{h}_l^{\text{cram}}(i)\| \, \|\mathbf{h}_l^{\text{clean}}(i)\|} \right) \tag{4}$$

where K is the number of layers used for alignment. We align the first K transformer layers (closest to the embedding layer).

The full objective combines reconstruction and alignment:

$$\mathcal{L} = \mathcal{L}_{\text{cram}} + \alpha \mathcal{L}_{\text{align}} \tag{5}$$

where α controls alignment strength.

4.3. Low-Dimensional Projection

We observed that optimization trajectories occupy low-dimensional subspaces (Section 5.1). This motivates constraining optimization to a learned projection:

$$\mathbf{e} = \mathbf{W}\mathbf{z} + \mathbf{b} \tag{6}$$

where $\mathbf{z} \in \mathbb{R}^k$ with $k \ll d$, $\mathbf{W} \in \mathbb{R}^{d \times k}$, and $\mathbf{b} \in \mathbb{R}^d$. For each sample, we randomly initialize $(\mathbf{W}, \mathbf{b}, \mathbf{z})$ and optimize them jointly, which restricts the compression embedding to a rank-k affine subspace.

After optimization, the effective embedding $\mathbf{e}$ can be materialized once and reused; the projection primarily changes the optimization geometry by introducing optimizer state (e.g., momentum) in the low-dimensional coordinates.

4.4. Attention Mass

To quantify attention hijacking (Section 5.5), we measure how much attention mass other positions allocate to the first position (position 0), which corresponds to the compression embedding in crammed inputs or the BOS token in the uncompressed baseline. Let $\mathbf{A}_l \in [0,1]^{S \times S}$ denote the attention matrix at layer l after averaging over heads. For a given prefix length $s \leq S$, we define the attention mass on position 0 as

$$m_l(s) = \frac{1}{s-1} \sum_{q=1}^{s-1} \mathbf{A}_l(q, 0) \tag{7}$$

which excludes self-attention ($q = 0$) and averages over query positions. We report $100 \cdot m_l(s)$ as a percentage, and average over a set of target prefix lengths $\mathcal{S}$:

$$\bar{m}_l = \frac{1}{|\mathcal{S}|} \sum_{s \in \mathcal{S}} m_l(s) \tag{8}$$

4.5. Information Gain

As a primary cramming metric we use information gain (Kuratov et al., 2025), defined as the reduction in total cross-entropy (in bits) on the target sequence when conditioning on the learned compression embedding. Let H_{LM} be the sum of per-token cross-entropies for the original token sequence, and let $H_{\text{comp}+LM}$ be the corresponding quantity when the model is conditioned on the compression embedding.

$$C_H = H_{LM} - H_{\text{comp}+LM} \tag{9}$$

Kuratov et al. (2025) show that cramming capacity is better characterized by information gain than by raw token count, and that this quantity is relatively stable across datasets.

5. Experiments

We evaluate progressive cramming on PG19 (Rae et al., 2020) across four model families: Pythia (Biderman et al.,

Table 1. Full vs. progressive cramming on PG19 (mean ± std over samples). "Full" cramming uses a fixed token budget and may stop below perfect reconstruction (Kuratov et al., 2025); progressive cramming fixes reconstruction at 100% and reports the achieved token count.

Type	Tokens	Info Gain	Accuracy
Llama-3.1-8B			
Full	1568	5002 ± 850	0.997 ± 0.003
Progr.	1064 ± 394	3028 ± 1321	1.0
Pythia1.4b			
Full	256	802 ± 113	0.985 ± 0.031
Progr.	544 ± 57	1694 ± 125	1.0

2023), Llama 3 (Dubey et al., 2024), SmolLM2 (allal et al., 2025), and Gemma 3 (Team et al., 2025).

Table 1 compares progressive cramming (targeting 100% reconstruction) to "full" cramming baselines that use fixed token budgets and may terminate below perfect accuracy. Because autoregressive decoding is brittle to early errors (Section 3.2), we interpret this comparison primarily through the lens of *perfect* reconstruction: progressive cramming trades a fixed-budget constraint for a fixed-accuracy constraint. In the default setting, progressive cramming underperforms full cramming in both information gain and reconstructed length. However, adding regularization via activation alignment and low-dimensional projection (Section 5.4) can recover capacity; for Llama-3.1-8B we reach 5103 ± 748 bits of information gain, comparable to 4865.7 ± 546.6 reported by Kuratov et al. (2025). A detailed comparison across additional model sizes can be found in Table 11 (Appendix C).

5.1. Optimization Trajectories

Progressive cramming enables tracking the optimization path through embedding space. We record the sequence of optimized embeddings $\{\mathbf{e}^{(k)}\}_{k=1}^{n}$ and analyze its geometry. For each sample, we run PCA on the stage embeddings $\{\mathbf{e}^{(k)}\}_{k=1}^{n}$ across k (using standard PCA mean-centering across stages) and define "PCA 99%" as the minimum number of components whose cumulative explained-variance ratio reaches 99%. We report this quantity averaged over samples. Across model families, trajectories are well-approximated by low-dimensional structure: a modest number of principal components explains 99% of the trajectory variance, even when hundreds (or thousands) of tokens are reconstructed (Table 2).

Notably, Gemma 3 achieves lower information gain at shorter reconstructed lengths than Llama and Pythia, consistent with stronger architectural or training-time constraints (e.g., logits softcapping (Team et al., 2024)). One plausible mechanism is that logit softcapping limits the model's ability to produce extremely peaked next-token distributions. Since cramming optimization implicitly pushes the model toward

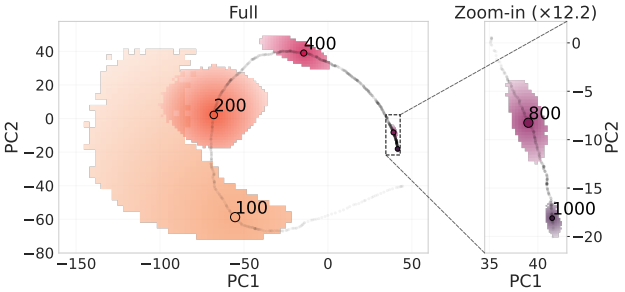

Figure 3. Progressive cramming trajectory for a length-1000 sequence on Llama3-8B, projected onto the first two PCA components. Each black point is the optimized compression embedding for a given prefix length. For prefix lengths $\{100, 200, 400, 800, 1000\}$, we additionally visualize the local accuracy landscape in this 2D plane (color saturation indicates higher reconstruction accuracy, capped at 90%). As sequence length increases, the basin of near-perfect reconstruction shrinks, making optimization harder. The first two PCA components explain 65.7% of the trajectory variance.

high-confidence, token-specific predictions over long horizons, bounding logit magnitudes can reduce the maximum separability between the correct token and close alternatives and can damp gradients in the high-confidence regime. Both effects may make it harder to reach perfect autoregressive reconstruction with a fixed optimization budget, yielding lower information gain and earlier saturation in reconstructed length.

Figure 3 visualizes a single trajectory projected onto the first two principal components. As the reconstructed prefix grows, the region of embeddings that attains near-perfect teacher-forced reconstruction contracts, suggesting that the optimization landscape becomes increasingly constrained at longer lengths.

Figure 4 shows that for Llama-3.1-8B, the number of components required to explain 99% variance grows sublinearly with prefix length and follows a slow (approximately logarithmic) trend across learning rates.

5.2. Low-Dimensional Projection

Low-dimensional projection (Section 4.3) changes the optimization geometry by restricting the learned embedding to an affine rank-k subspace. Empirically, this can improve stability and alter the effective capacity:

- It often reduces sensitivity to learning rate by concentrating optimization into a smaller coordinate system.
- It can reduce the intrinsic dimensionality of trajectories (PCA 99%) while preserving, or sometimes increasing, information gain.

Table 2 compares baseline progressive cramming to a rep-

Table 2. Progressive cramming variants and key metrics across model families. For each family we report baseline progressive cramming, low-dimensional projection only ("dim"), activation alignment only (α, L), and their combination.

Model	Compressed Tokens	Information Gain	Trajectory Length	PCA 99%
Llama-3.1-8B lr=0.1	1437.6 ± 380.1	4391 ± 1408	6174 ± 1263	82.78 ± 9.07
Llama-3.1-8B dim=256 lr=0.1	1697.2 ± 304.2	4814 ± 1731	243424 ± 57429	58.58 ± 6.4
Llama-3.1-8B lr=0.1 $\alpha = 1.0$ $L = 4$	1795.1 ± 225.7	5605 ± 596	7525 ± 774	81.26 ± 3.76
Llama-3.1-8B dim=256 lr=0.1 $\alpha = 1.0$ $L = 8$	1732.7 ± 253.2	4009 ± 2414	226998 ± 45513	62.38 ± 5.08
pythia-1.4b lr=0.5	430.4 ± 64.9	1639 ± 188	6496 ± 610	50.06 ± 3.31
pythia-1.4b lr=0.5 $\alpha = 1.0$ $L = 8$	397.7 ± 57	1513 ± 163	6187 ± 513	46.44 ± 2.74
pythia-1.4b dim=256 lr=0.5	500.2 ± 74.8	1874 ± 283	598025 ± 725213	67.58 ± 19.82
pythia-1.4b dim=256 lr=0.5 $\alpha = 1.0$ $L = 8$	405.3 ± 82.5	1521 ± 276	1269523 ± 618392	58.92 ± 15.36
SmolLM2-1.7B lr=0.1	338.3 ± 69.1	1213 ± 166	1056 ± 153	33.16 ± 2.76
SmolLM2-1.7B lr=0.1 $\alpha = 1.0$ $L = 8$	242.6 ± 63.2	880 ± 186	894 ± 121	26.46 ± 2.64
SmolLM2-1.7B dim=256 lr=0.1	957.4 ± 142.5	3271 ± 309	132821 ± 44546	30.94 ± 4.93
SmolLM2-1.7B dim=256 lr=0.1 $\alpha = 1.0$ $L = 8$	918.6 ± 199.1	3135 ± 628	142874 ± 54379	33.28 ± 6.69
gemma-3-4b-pt lr=0.1	213.6 ± 109.2	783 ± 370	910 ± 261	30.8 ± 9.45
gemma-3-4b-pt lr=0.1 $\alpha = 1.0$ $L = 8$	202 ± 119.4	763 ± 425	869 ± 275	26.86 ± 7.7
gemma-3-4b-pt dim=32 lr=0.1	697.4 ± 165.3	2141 ± 786	27948 ± 15706	22.5 ± 5.14
gemma-3-4b-pt dim=32 lr=0.1 $\alpha = 1.0$ $L = 8$	680.7 ± 221	2091 ± 730	35384 ± 29987	21.78 ± 4.94

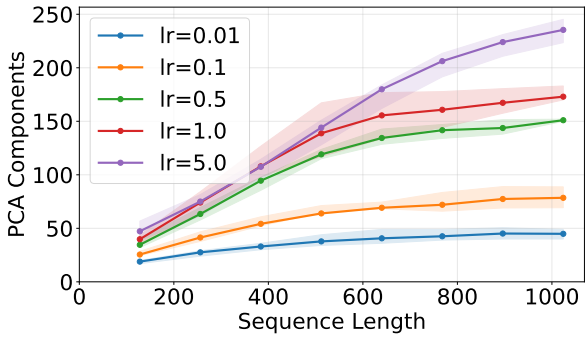

Figure 4. Number of principal components required to explain 99% of progressive trajectory variance vs. reconstructed prefix length for Llama-3.1-8B, shown for multiple learning rates.

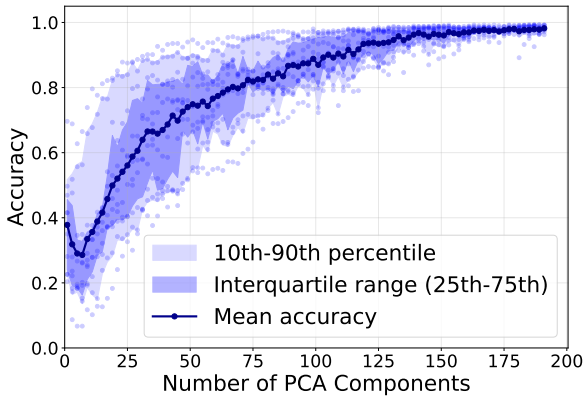

Figure 5. Teacher-forced reconstruction accuracy for PCA-reconstructed compression embeddings vs. number of PCA components (Llama-3.1-8B).

resentative low-dimensional projection setting ("dim") for each family. Two takeaways stand out:

- For Llama-3.1-8B and SmolLM2-1.7B, projection can substantially increase achievable token counts and information gain compared to the unconstrained baseline.
- For Pythia-1.4B projection can reduce token count and information gain relative to the baseline; combining projection with activation alignment partially recovers performance compared to projection alone.

5.3. PCA Reconstruction

We evaluate reconstruction accuracy when progressive embeddings are reconstructed from PCA components. Figure 5 reports accuracy as a function of the number of components. Although Table 2 suggests that for Llama-3.1-8B roughly 74 PCA components explain 99% of trajectory variance, achieving near-perfect teacher-forced reconstruction requires substantially more components. We also observe the same failure mode as in the full cramming setup (Section 3.2): errors cluster at the start of the sequence (often at the first decoded token), which then propagates and breaks subsequent autoregressive decoding. This behavior appears whenever reconstruction accuracy is below 100%; with perfect reconstruction, this failure mode disappears.

5.4. Activation Alignment

Table 2 studies activation alignment (defined in Section 4.2) and its interaction with low-dimensional projection. Alignment acts as a regularizer: it can significantly reduce trajectory length and PCA 99% (indicating smoother, more constrained optimization), but it may trade off against raw

cramming capacity depending on the model family. In particular, for Llama-3.1-8B, alignment increases information gain and token count relative to the baseline, whereas for SmolLM2-1.7B and Pythia-1.4B alignment alone reduces capacity but can be combined with projection to recover part of the loss. Across settings, we observe that alignment qualitatively changes attention behavior, often pushing attention hijacking effects toward deeper layers.

5.5. Attention Hijacking

Perfect reconstruction can arise from qualitatively different mechanisms, ranging from faithful information encoding to brittle steering; to probe the mechanism, we measure whether the compression embedding becomes an *attention sink* that captures a disproportionate fraction of attention, analogous to (and potentially competing with) the BOS token. We quantify attention sinks using the attention-mass metric from Section 4.4, comparing attention directed to position 0 in the crammed setting (compression embedding) against attention directed to position 0 in the uncompressed baseline (BOS). We refer to this phenomenon as *attention hijacking* to emphasize that strong attention capture is associated with a breakdown of downstream language modeling capabilities (Section 5.6).

Table 3 shows that attention hijacking is highly model-dependent when aggregated over layers. SmolLM2 and Pythia exhibit strong correlation between the layer-wise attention-mass profiles for the BOS token and the compression embedding. In these families, the compression embedding attracts substantial attention mass in the same layers where BOS is an attention sink; in Pythia it can match or exceed BOS attention on average. This suggests that the compression embedding may function as an attention sink whose attention capture coincides with degraded downstream behavior (Section 5.6).

In contrast, Llama and Gemma show much lower compression-embedding attention mass than BOS on average and weak (or negative) correlation, suggesting that hijacking concentrates in specific layers rather than uniformly matching the BOS pattern.

Motivated by this similarity, we test whether including both a BOS token and a compression embedding in the prompt creates interference between two attention sinks. We therefore run ablations that remove the BOS token and recompute progressive cramming metrics (Table 4). Removing BOS increases reconstructed length and information gain (and reduces compression-embedding attention mass) for Llama-3.1-8B, but has little effect on the other model families.

5.6. Downstream Evaluation

We test whether cramming preserves the *useful* information in a prefix by evaluating standard multiple-choice benchmarks (HellaSwag and ARC) under likelihood-based scoring. These benchmarks use relatively short contexts, so they isolate capability failures that occur even when the task does not require long-range context retention. If the compression embedding faithfully encodes the prefix in a way the model can consume, accuracy should remain close to the uncompressed baseline; large drops indicate that conditioning on the compression embedding disrupts the model's ability to leverage the prefix for downstream reasoning.

Setup. We evaluate on HellaSwag and ARC under two conditions:

1. Original prefix (baseline)
2. Compression embedding + original prefix

For the second condition, we optimize a separate compression embedding per benchmark instance, targeting perfect teacher-forced reconstruction of the corresponding prefix within a fixed optimization budget. We use the standard full cramming setup (a single embedding optimized for the entire prefix), rather than the progressive stage-wise procedure from Section 4.1. In practice, this optimization does not guarantee perfect reconstruction for every benchmark instance; we do not filter out imperfectly reconstructed cases, so reported downstream accuracy reflects both the effect of conditioning on the learned embedding and any residual reconstruction errors. Each benchmark instance provides four candidate continuations; we select the continuation with the lowest normalized negative log-likelihood (per suffix token). Under this 4-way multiple-choice protocol, random guessing yields 25% accuracy.

Results. Table 5 shows that prepending a crammed compression embedding consistently reduces downstream accuracy across model families, often collapsing to (or below) random guessing. Together with the attention analysis, these results suggest that high reconstruction performance can coincide with brittle steering (attention hijacking) rather than a transferable semantic representation. This highlights a key limitation of using reconstruction accuracy alone: perfect reconstruction is insufficient, and future work must explicitly evaluate capability preservation in any downstream semantic evaluation and broader capability benchmarks overall.

6. Conclusion

We introduced progressive cramming, which grows the reconstruction target token-by-token and enables controlled measurement of when embedding-based reconstruc-

Table 3. Attention hijacking summary. "Compression Embedding" is attention mass to position 0 for the compression embedding in the crammed input; "BOS Token" is attention mass to position 0 for the uncompressed baseline; "Diff" is (Compression−BOS). "Correlation" is the Pearson correlation between per-layer attention-mass profiles. ~~B~~ stands for removed BOS token for cramming compression column.

Model	Compression Embedding (%)	BOS Token Original (%)	Diff (%)	Correlation
Llama-3.1-8B lr=0.1	38.07 ± 17.98	70.56 ± 1.26	-32.49 ± 18.96	0.4104
Llama-3.1-8B ~~B~~ lr=0.1	18.93 ± 10.43	70.05 ± 1.29	-51.12 ± 10.33	0.4688
pythia-1.4b lr=0.5	37.41 ± 4.12	24.03 ± 2.49	13.37 ± 5.17	0.8315
pythia-1.4b ~~B~~ lr=0.5	38 ± 4.72	24.74 ± 2.49	13.27 ± 5.89	0.8454
SmolLM2-1.7B lr=0.1	49.85 ± 2.75	56.47 ± 9.19	-6.62 ± 9.32	0.9532
SmolLM2-1.7B ~~B~~ lr=0.1	49.95 ± 2.82	56.47 ± 9.19	-6.52 ± 9.44	0.9535
gemma-3-4b-pt lr=0.1	7.97 ± 2.09	61.17 ± 1.02	-53.19 ± 2.05	0.0497
gemma-3-4b-pt ~~B~~ lr=0.1	6.84 ± 0.4	63.56 ± 1.02	-56.72 ± 1.22	-0.1461

Table 4. Progressive cramming with and without the BOS token. ~~B~~ denotes runs where the BOS token is removed.

Model	Cram Tokens	Info Gain
Llama-3.1-8B lr=0.1	1064 ± 394	3028 ± 1321
Llama-3.1-8B ~~B~~ lr=0.1	1412 ± 320	4473 ± 1035
pythia-1.4b lr=0.5	544 ± 57	1694 ± 125
pythia-1.4b ~~B~~ lr=0.5	541 ± 68	1687 ± 121
SmolLM2-1.7B lr=0.1	370 ± 113	1119 ± 350
SmolLM2-1.7B ~~B~~ lr=0.1	361 ± 105	1106 ± 328
gemma-3-4b-pt lr=0.1	286 ± 165	949 ± 466
gemma-3-4b-pt ~~B~~ lr=0.1	207 ± 86	1196 ± 445

Table 5. Downstream multiple-choice evaluation on HellaSwag and ARC-Easy (4-way; chance = 25%). "Base" uses the original prefix; "Cram" prepends a per-instance optimized compression embedding in the full cramming setup (reconstruction is targeted but not guaranteed to be perfect). ~~B~~ denotes experiments with the BOS token removed.

	HellaSwag		**ARC-E**	
Model	Base	Cram	Base	Cram
Llama-3.1-8B	40.79%	23.24%	38.48%	27.87%
Llama-3.1-8B ~~B~~	38.84%	22.30%	37.69%	29.34%
pythia-1.4b	43.06%	26.32%	45.95%	24.32%
pythia-1.4b ~~B~~	43.06%	26.32%	45.95%	24.32%
SLM2-1.7B	51.54%	23.17%	58.77%	30.69%
SLM2-1.7B ~~B~~	51.54%	23.17%	58.77%	30.69%
gemma-3-4b	29.27%	26.18%	39.70%	24.29%
gemma-3-4b ~~B~~	28.84%	26.18%	34.62%	24.16%

tion succeeds or fails under a fixed optimization budget. Across model families, progressive trajectories are low-dimensional, and attention-mass analyses show that crammed embeddings can become intermediate-layer attention sinks, consistent with attention hijacking. Critically, downstream evaluation indicates that optimizing embeddings for reconstruction can coincide with large capability drops, even when the original prefix remains in context, so reconstruction accuracy alone is not a sufficient criterion for meaningful compression.

Our study remains limited in scope: we use a small number of PG19 samples, and our mechanistic probes are correlational rather than causal. Future work should strengthen causal analysis via targeted interventions, and develop a *fast* compressor that produces semantically meaningful compression embeddings.

Impact Statement

This paper presents work whose goal is to advance the field of Machine Learning. There are many potential societal consequences of our work, none which we feel must be specifically highlighted here.

7. Reviewer 1 (neRn)

We thank the reviewer for the thorough evaluation and constructive suggestions. We address each point below.

W1. Limited number of samples.

We reproduced the data preparation pipeline from Kuratov et al. and re-ran all experiments on the same 50 samples used in the original full cramming paper. Our main conclusions — low PCA dimensionality and higher Information Gain compared to full cramming — remain valid on this data.

However, the expanded evaluation revealed an important finding: for all models *except* Llama-3.1-8B, activation alignment leads to significant degradation in both compressed tokens and Information Gain. The best results across all experiments are achieved with **low-dimensional linear projection alone**. Only Llama-3.1-8B shows competitive performance when combining low-dimensional projection with activation alignment; for the remaining models (Pythia-1.4B, SmolLM2-1.7B, Gemma-3-4B-pt) the combination does not improve over projection-only variants.

We present the updated version of progressive cramming table Table 6 reporting progressive cramming variants and key metrics across all four model families on the 50-sample subset.

W2. Causality analysis for "attention hijacking" and "degraded downstream performance"

We thank the reviewer for this suggestion. We conducted causal intervention experiments using **attention knockout**—masking pre-softmax logits corresponding to the compression embedding to $-\infty$ at selected layers, thereby completely removing its influence on the residual stream. Three protocols were used: (1) **per-layer knockout** (masking at a single layer), (2) **forward cumulative knockout** (masking layers 0 through k), and (3) **reverse cumulative knockout** (masking layers $L-1$ down to k). Each condition is evaluated on reconstruction accuracy and downstream capability (HellaSwag).

The results reveal that **attention mass and causal importance are dissociated**:

- Per-layer knockout at early layers degrades reconstruction while slightly *improving* downstream accuracy, indicating that early layers causally drive the downstream disruption.
- Forward cumulative knockout rapidly restores downstream accuracy once the first several layers are masked
- Reverse cumulative knockout does not recover downstream performance until masking reaches the early layers.

This **forward/reverse asymmetry** is the core causal argument: downstream collapse is driven by early-layer interactions with the compression embedding. Late layers, despite exhibiting the highest attention mass, have minimal causal impact on downstream performance when knocked out — the observed concentration is a *symptom* rather than the cause.

Questions

Q1. Limited number od samples. See **W1**.

Q2. Causality analysis for attention hijacking. See **W2**.

Q3. Downstream evaluation with progressive cramming.

We present downstream evaluation results for progressive cramming in Table 7. For progressive cramming, we report metrics computed exclusively over fully converged samples (i.e., those achieving perfect reconstruction). The performance drop relative to the baseline is comparable for both full and progressive cramming across all models, ranging from approximately 34%–38%.

Note that Gemma-3-4B shows 0% convergence under progressive cramming, meaning no sample achieved perfect token-by-token reconstruction. Notably, full cramming still yields a relatively small performance drop for Gemma (54.97% vs. 57.07%). This finding requires further research.

Erratum: Downstream evaluation bug

We identified a bug in the perplexity computation for the HellaSwag and ARC benchmarks in the originally submitted version. Specifically, the compression embedding introduced an off-by-one token shift that was not accounted for during likelihood scoring, resulting in the near-random accuracy reported in the original paper. The corrected results are shown above.

8. Reviewer 2 (eppq)

We appreciate the reviewer's careful reading and valuable feedback. Below we respond to each concern.

Weaknesses:

W1. Limited number of samples.

We agree that 10 samples provide limited statistical reliability. Following this suggestion (and a similar concern raised by another reviewer), we re-ran all experiments on the 50-sample subset used by Kuratov et al. in the original full cramming paper. Our main conclusions—low PCA dimensionality and higher Information Gain of progressive cramming compared to full cramming—remain stable at this scale. The updated results are presented in Table 6.

Also see **W1** in our answer to reviewer `neRn`.

Table 6. Progressive cramming variants and key metrics across model families on **50 samples**. For each family we report baseline progressive cramming, low-dimensional projection only ("dim"), activation alignment only (α, L), and their combination.

Model	Compressed Tokens	Information Gain	Trajectory Length	PCA 99%
Llama-3.1-8B lr=0.1	1437.6 ± 380.1	4391 ± 1408	6174 ± 1263	82.78 ± 9.07
Llama-3.1-8B lr=0.1 $\alpha = 1.0$ $L = 4$	1795.1 ± 225.7	5605 ± 596	7525 ± 774	81.26 ± 3.76
Llama-3.1-8B dim=256 lr=0.1	1697.2 ± 304.2	4814 ± 1731	243424 ± 57429	58.58 ± 6.4
Llama-3.1-8B dim=256 lr=0.1 $\alpha = 1.0$ $L = 8$	1732.7 ± 253.2	4009 ± 2414	226998 ± 45513	62.38 ± 5.08
pythia-1.4b lr=0.5	430.4 ± 64.9	1639 ± 188	6496 ± 610	50.06 ± 3.31
pythia-1.4b lr=0.5 $\alpha = 1.0$ $L = 8$	397.7 ± 57	1513 ± 163	6187 ± 513	46.44 ± 2.74
pythia-1.4b dim=256 lr=0.5	500.2 ± 74.8	1874 ± 283	598025 ± 725213	67.58 ± 19.82
pythia-1.4b dim=256 lr=0.5 $\alpha = 1.0$ $L = 8$	405.3 ± 82.5	1521 ± 276	1269523 ± 618392	58.92 ± 15.36
SmolLM2-1.7B lr=0.1	338.3 ± 69.1	1213 ± 166	1056 ± 153	33.16 ± 2.76
SmolLM2-1.7B lr=0.1 $\alpha = 1.0$ $L = 8$	242.6 ± 63.2	880 ± 186	894 ± 121	26.46 ± 2.64
SmolLM2-1.7B dim=256 lr=0.1	957.4 ± 142.5	3271 ± 309	132821 ± 44546	30.94 ± 4.93
SmolLM2-1.7B dim=256 lr=0.1 $\alpha = 1.0$ $L = 8$	918.6 ± 199.1	3135 ± 628	142874 ± 54379	33.28 ± 6.69
gemma-3-4b-pt lr=0.1	213.6 ± 109.2	783 ± 370	910 ± 261	30.8 ± 9.45
gemma-3-4b-pt lr=0.1 $\alpha = 1.0$ $L = 8$	202 ± 119.4	763 ± 425	869 ± 275	26.86 ± 7.7
gemma-3-4b-pt dim=32 lr=0.1	697.4 ± 165.3	2141 ± 786	27948 ± 15706	22.5 ± 5.14
gemma-3-4b-pt dim=32 lr=0.1 $\alpha = 1.0$ $L = 8$	680.7 ± 221	2091 ± 730	35384 ± 29987	21.78 ± 4.94

Table 7. Hellaswag Downstream Evaluation results For Progressive Cramming

Model	Baseline Acc	Full Acc	Progressive Acc	Progressive Converged
Llama-3.1-8B	61.91%	36.80%	38.62%	97.66%
SmolLM2-1.7B	53.56%	36.90%	34.70%	95.51%
gemma-3-4b-pt	57.07%	54.97%	0.00%	0.00%
pythia-1.4b	44.00%	35.08%	35.12%	97.07%

W2. Imperfect compression for HellaSwag and ARC evaluation

To disentangle the effect of imperfect reconstruction from the effect of the compression embedding itself, we emphasize the difference between autoregressive generation and multiple-choice benchmarks such as HellaSwag and ARC. In greedy generation, even small reconstruction errors can lead to large deviations in the output due to error accumulation, especially when errors occur early in the sequence.

In contrast, HellaSwag and ARC operate in a non-generative (scoring) regime: the model evaluates fixed context–continuation pairs without regenerating the prefix. As a result, these benchmarks are significantly less sensitive to small reconstruction imperfections, since no sequential error propagation occurs.

The near-identical token-normalized accuracies for SmolLM2-1.7B (Table 8) indicate that imperfect reconstruction has minimal impact on benchmark performance. This confirms that the observed degradation is driven primarily by the compression embedding itself, rather than by reconstruction errors.

W3. Narrow downstream evaluation relying only on HellaSwag and ARC

We agree that HellaSwag and ARC alone probe a limited slice of downstream behavior. To address this, we additionally evaluated on **5-shot MMLU** (512 samples) under three compression modes: *few_shot* (only few-shot examples are compressed), *full_prefix* (both few-shot examples and the question are compressed), and *random* (a random embedding replaces the compression embedding as a control).

Table 8. Token-normalized accuracy (%) for HellaSwag, ARC-Easy, and ARC-Challenge on SmolLM2-1.7B. Values are reported for perfectly reconstructed examples and for all examples.

Subset	**HellaSwag**	**ARC-Easy**	**ARC-Challenge**
Perfect only	36.05	55.39	28.86
All samples	36.14	55.21	28.44

The results Table 9 are consistent with our earlier findings. When only few-shot tokens are compressed, all models shows accuracy degradation (ranging from -2% for Gemma-3-4B to -15% for Llama-3.1-8B), confirming that the model retains generative capability when uncompressed tokens follow the compression embedding. In contrast, *full_prefix* compression—where compression embedding compresses full prefix—leads to **complete failure**: accuracy drops to near zero and the models produce no valid (parseable) answers, indicating total collapse of the generative process. Notably, the *random* baseline causes negligible degradation ($\leq 1\%$), showing that the mere presence of

an out-of-distribution token is not sufficient to explain the failure—it is specifically the *optimized* compression embedding that disrupts downstream computation.

These results on a generative benchmark with longer context reinforce the conclusions drawn from HellaSwag and ARC, and demonstrate that the downstream collapse generalizes beyond multiple-choice likelihood-based evaluation.

W4. Compute expenditure vs. statistical coverage

The reported 2800 GPU-hours represent *total* compute including all debugging, hyperparameter sweeps, and failed runs—not the cost of final experiments alone.

Progressive cramming is inherently expensive: current implementation was optimized through dynamic batching and code optimization, but it remains roughly $2\times$ slower than full cramming.

As noted in our response to reviewer `neRn` to W1, we have now extended all experiments to 50 samples following Kuratov et al., and the main conclusions remain stable Table 6.

W5. Paper length and organization

We agree that the main body can be made more substantial. In the revised version we will reorganize the paper, ensuring the paper makes full use of the available page budget.

Erratum: Downstream evaluation bug

After submission, we discovered a bug in the perplexity computation for the HellaSwag and ARC benchmarks. The compression embedding introduced an off-by-one token shift that was not accounted for during likelihood scoring, causing the near-random downstream accuracy reported in the original paper. After correction, HellaSwag accuracy under cramming ranges from 34%–38% for most models, representing a consistent but moderate drop from baseline—in line with our central claim that high reconstruction accuracy does not imply preservation of downstream-relevant semantics. Corrected results are presented in Table 7. See also response to Reviewer `neRn`, Q3.

9. Reviewer 3 (UoVE)

We are grateful for the detailed and insightful review. We address the raised concerns point by point.

W1. Attention-hijacking narrative

We appreciate this critique and agree that the original presentation mixed observations with mechanistic claims insufficiently supported by causal evidence. We address this in two ways.

Causal evidence. We conducted attention knockout experiments (see answer to reviewer `neRn`, W2) that provide direct causal support: the forward/reverse knockout asymmetry shows that early-layer interactions with the compression embedding drive downstream degradation, while late-layer attention mass has minimal causal impact. MMLU evaluation (see answer to reviewer `eppq`, W3) further confirms that full-prefix compression causes complete generative failure.

Model-family dependence. We agree that a single "competing attention sinks" explanation is too strong. Different architectures handle BOS tokens and attention sinks differently (e.g., Llama uses a dedicated BOS sink, while Pythia distributes attention more uniformly), so the *mechanism* varies even though the *downstream effect* is consistent. In the revision we will restructure Section 4.4, leading with causal knockout results and repositioning attention-mass statistics and BOS ablation as supporting model-specific observations.

Erratum: Downstream evaluation bug

After submission, we discovered a bug in the perplexity computation for the HellaSwag and ARC benchmarks. The compression embedding introduced an off-by-one token shift that was not accounted for during likelihood scoring, causing the near-random downstream accuracy reported in the original paper. After correction, HellaSwag accuracy under cramming ranges from 34%–38% for most models, representing a consistent but moderate drop from baseline—in line with our central claim that high reconstruction accuracy does not imply preservation of downstream-relevant semantics. Corrected results are presented in Table 7. See also response to Reviewer `neRn`, Q3.

10. Reviewer 4 (sozN)

We thank the reviewer for the thoughtful comments and suggestions for strengthening the paper. Our responses follow below.

Weaknesses:

W1. Single evaluation dataset.

To address this issue we reproduced fanfics dataset from Kuratov et al and runned experiments on it. Results may be found in Table 11.

The key findings are consistent with those observed on PG-19. Information gain follows the same patterns across configurations: low-dimensional projection substantially increases both the number of compressed tokens and information gain.

The low-dimensional structure of compression trajectories also generalizes to Fanfics data. While PCA 99% values are somewhat larger than those observed on PG-19—likely reflecting the greater distributional diversity of fan-fiction text—they remain at least an order of magnitude below the embedding dimensionality.

Table 9. 5-shot MMLU accuracy results on 512 samples with compression embedding in context. **CompMode = few_shot** - only few-shot tokens were compressed with cramming. **CompMode = full_prefix** - few-shot tokens and question tokens were compressed. **CompMode = random** - random embedding was placed instead of compression embedding. **Baseline Acc** - accuracy for base model without compression embedding. **Compressed Acc** - accuracy for model with compression embedding in context (regular tokens were placed after compression embedding). **Valid** columns are stands for percent of valid generated answers (when answer was parsed successfully).

Model	CompMode	Baseline Acc	Compressed Acc	Acc Diff	Baseline Valid	Compressed Valid
Llama-3.1-8B	few_shot	63.22%	48.14%	-15.08%	100.00%	100.00%
Llama-3.1-8B	full_prefix	63.22%	0.00%	-63.22%	100.00%	0.00%
Llama-3.1-8B	random	63.22%	62.71%	-0.51%	100.00%	100.00%
SmolLM2-1.7B	few_shot	50.98%	45.91%	-5.07%	100.00%	100.00%
SmolLM2-1.7B	full_prefix	50.98%	0.00%	-50.98%	100.00%	0.00%
SmolLM2-1.7B	random	50.98%	46.66%	-4.32%	100.00%	100.00%
gemma-3-4b-pt	few_shot	58.13%	56.07%	-2.05%	100.00%	100.00%
gemma-3-4b-pt	full_prefix	58.13%	0.00%	-58.13%	100.00%	0.00%
gemma-3-4b-pt	random	58.13%	59.18%	1.05%	100.00%	100.00%
pythia-1.4b	few_shot	28.15%	21.27%	-6.88%	100.00%	95.70%
pythia-1.4b	full_prefix	28.15%	4.83%	-23.33%	100.00%	24.80%
pythia-1.4b	random	28.15%	27.01%	-1.14%	100.00%	100.00%

Table 10. Progressive cramming variants and key metrics across model families on **50 samples** of **Fanfics** dataset following Kuratov et al. For each family we report baseline progressive cramming, low-dimensional projection only ("dim"), activation alignment only (α, L), and their combination.

Model	Compressed Tokens	Information Gain	Trajectory Length	PCA 99%
pythia-1.4b lr=0.5	564.5 ± 7.5	1255 ± 309	6331 ± 739	46.12 ± 4.17
pythia-1.4b lr=0.5 $\alpha = 1.0$ $L = 8$	517 ± 26	1176 ± 295	6180 ± 598	43.06 ± 3.49
pythia-1.4b dim=256 lr=0.5	619.5 ± 12.5	2083 ± 258	510014 ± 92005	124.5 ± 14.23
pythia-1.4b dim=256 lr=0.5 $\alpha = 1.0$ $L = 8$	540 ± 54	1693 ± 181	494306 ± 58027	128.8 ± 12.6
SmolLM2-1.7B lr=0.1	486 ± 45	710 ± 666	1038 ± 139	28.72 ± 4.68
SmolLM2-1.7B lr=0.1 $\alpha = 1.0$ $L = 8$	337.5 ± 19.5	640 ± 271	878 ± 137	22.76 ± 4.33
SmolLM2-1.7B dim=256 lr=0.1	1050 ± 72	3318 ± 659	321418 ± 59468	95.14 ± 16.53
SmolLM2-1.7B dim=256 lr=0.1 $\alpha = 1.0$ $L = 8$	968 ± 1	3396 ± 342	305391 ± 53905	71.44 ± 12.46
gemma-3-4b-pt lr=0.1	416.4 ± 22.3	-118 ± 1241	870 ± 274	24.22 ± 10.07
gemma-3-4b-pt lr=0.1 $\alpha = 1.0$ $L = 8$	410.4 ± 43.7	437 ± 509	868 ± 305	23.02 ± 9.62
gemma-3-4b-pt dim=32 lr=0.1	907.8 ± 90.8	2979 ± 801	84162 ± 15369	39.44 ± 3.59
gemma-3-4b-pt dim=32 lr=0.1 $\alpha = 1.0$ $L = 8$	962 ± 31.3	2667 ± 1248	111260 ± 23499	43.86 ± 3.81

W2. No proposal for introducing valuable semantic properties

We agree—cramming optimizes solely for token reconstruction and does not explicitly target semantic properties. Introducing such properties is outside the scope of this work. Compression approaches that do optimize for semantics (e.g., Gisting, Activation Beacon, AutoCompressors, ICAE) are discussed in the related work section and represent a complementary research direction.

W3. Higher cost of progressive optimization procedure

Progressive cramming optimization is $2\times$ slower than full cramming. The main bottleneck in current implementation is that within a batch, a single slowly converging sample blocks new token addition for all other samples.

Questions:

Q1. How does the speed (in terms of optimization steps) of progressive optimization relate to vanilla cramming?

Full cramming was limited to 10k steps. Progressive cramming, due to its autoregressive approach, may finish earlier. On average, optimization steps remain below 10k for simple cramming:

- Pythia-1.4B: 6844 ± 2043
- SmolLM2-1.7B: 9391 ± 2068
- Gemma-3-4B-pt: 8954 ± 2900

Experiments for Llama 8B were not conducted due to time constraints.

Q2. How semantic properties of the resulting representations can be evaluated other than by concatenation with the queries?

Generative benchmark. We additionally report evaluation results on **5-shot MMLU** in Table 9. Unlike HellaSwag/ARC, which are likelihood-based (scoring pre-defined completions), MMLU is generative—the model must produce an answer. Compressing only few-shot examples causes modest degradation (-2% to -15%), while compressing the full prefix leads to complete failure (0% valid answers), consistent with our findings on likelihood-based benchmarks. See also answer to reviewer `eppq`, W3.

Different perplexity computation strategies. We evaluated the semantic properties of compression embeddings on the **HellaSwag** and **ARC** benchmarks using different perplexity computation strategies. Specifically, we varied which token logits were included when computing perplexity:

- **baseline** – standard perplexity without compression embeddings; both context and answer ending logits are included.
- **baseline_endings** – standard perplexity without compression embeddings; only the answer ending logits are considered.
- **compression** – perplexity computed with compression embeddings; both context and answer ending logits are included.
- **compression_endings** – perplexity computed with compression embeddings; only the answer ending logits are considered.
- **compression_only** – perplexity computed using only compression embeddings in place of the context; only the answer ending logits are included.
- **compression_only_endings** – perplexity computed using only compression embeddings in place of the context; only the answer ending logits are included.

These strategies allow us to assess how much semantic information is preserved in the compression embeddings under different conditions. By comparing performance across these methods, we can isolate the contributions of context versus compression tokens and evaluate the robustness of semantic representations in compressed form (see Table 10).

Q3. How can the number of optimization steps be reduced?

Our low-dimensional projection partially addresses this—it converges faster and compresses fewer tokens. Further reduction could involve training an encoder to predict a good starting point, left for future work.

Erratum: Downstream evaluation bug

After submission, we discovered a bug in the perplexity computation for the HellaSwag and ARC benchmarks. The compression embedding introduced an off-by-one token shift that was not accounted for during likelihood scoring, causing the near-random downstream accuracy reported in the original paper. After correction, HellaSwag accuracy under cramming ranges from 34%–38% for most models, representing a consistent but moderate drop from baseline—in line with our central claim that high reconstruction accuracy does not imply preservation of downstream-relevant semantics. Corrected results are presented in Table 7. See also response to Reviewer `neRn`, Q3.

Table 10. Token-normalized accuracy (%) on HellaSwag, ARC-Easy, and ARC-Challenge for SmolLM2-1.7B under different perplexity computation strategies (perfectly reconstructed samples).

Setup	**HellaSwag**	**ARC-Easy**	**ARC-Challenge**
Baseline	52.41	68.72	36.66
Baseline endings	53.30	53.88	40.29
Compression	36.47	55.87	28.62
Compression endings	40.70	41.63	31.36
Compression only	34.57	30.92	22.94

References

allal, L. B., Lozhkov, A., Bakouch, E., Blazquez, G. M., Penedo, G., Tunstall, L., Marafioti, A., Lajarín, A. P., Kydlíček, H., Srivastav, V., Lochner, J., Fahlgren, C., NGUYEN, X. S., Burtenshaw, B., Fourrier, C., Zhao, H., Larcher, H., Morlon, M., Zakka, C., Raffel, C., Werra, L. V., and Wolf, T. SmolLM2: When smol goes big — data-centric training of a fully open small language model. In *Second Conference on Language Modeling*, 2025. URL https://openreview.net/forum?id=3JiCl2A14H.

Biderman, S., Schoelkopf, H., Anthony, Q. G., Bradley, H., O'Brien, K., Hallahan, E., Khan, M. A., Purohit, S., Prashanth, U. S., Raff, E., et al. Pythia: A suite for analyzing large language models across training and scaling. In *International Conference on Machine Learning*, pp. 2397–2430. PMLR, 2023.

Chevalier, A., Wettig, A., Ajith, A., and Chen, D. Adapting language models to compress contexts. In Bouamor, H., Pino, J., and Bali, K. (eds.), *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 3829–3846, Singapore, December 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.emnlp-main.232. URL https://aclanthology.org/2023.emnlp-main.232/.

Dubey, A., Jauhri, A., Pandey, A., Kadian, A., Al-Dahle, A., Letman, A., Mathur, A., Schelten, A., Yang, A., Fan, A., et al. The llama 3 herd of models. *arXiv e-prints*, pp. arXiv–2407, 2024.

Ge, T., Jing, H., Wang, L., Wang, X., Chen, S.-Q., and Wei, F. In-context autoencoder for context compression in a large language model. In *The Twelfth International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=uREj4ZuGJE.

Kuratov, Y., Arkhipov, M., Bulatov, A., and Burtsev, M. Cramming 1568 tokens into a single vector and back again: Exploring the limits of embedding space capacity. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 19323–19339, 2025.

Lester, B., Al-Rfou, R., and Constant, N. The power of scale for parameter-efficient prompt tuning. In Moens, M.-F., Huang, X., Specia, L., and Yih, S. W.-t. (eds.), *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pp. 3045–3059, Online and Punta Cana, Dominican Republic, November 2021. Association for Computational Linguistics. doi: 10.18653/v1/2021.emnlp-main.243. URL https://aclanthology.org/2021.emnlp-main.243/.

Li, X. L. and Liang, P. Prefix-tuning: Optimizing continuous prompts for generation. In Zong, C., Xia, F., Li, W., and Navigli, R. (eds.), *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pp. 4582–4597, Online, August 2021. Association for Computational Linguistics. doi: 10.18653/v1/2021.acl-long.353. URL https://aclanthology.org/2021.acl-long.353/.

Mezentsev, G. and Oseledets, I. Exploring the hidden capacity of llms for one-step text generation. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pp. 22891–22900, 2025.

Rae, J. W., Potapenko, A., Jayakumar, S. M., Hillier, C., and Lillicrap, T. P. Compressive transformers for long-range sequence modelling. In *International Conference on Learning Representations*, 2020. URL https://openreview.net/forum?id=SylKikSYDH.

Team, G., Riviere, M., Pathak, S., Sessa, P. G., Hardin, C., Bhupatiraju, S., Hussenot, L., Mesnard, T., Shahriari, B., Ramé, A., Ferret, J., Liu, P., Tafti, P., Friesen, A., Casbon, M., Ramos, S., Kumar, R., Lan, C. L., Jerome, S., Tsitsulin, A., Vieillard, N., Stanczyk, P., Girgin, S., Momchev, N., Hoffman, M., Thakoor, S., Grill, J.-B., Neyshabur, B., Bachem, O., Walton, A., Severyn, A., Parrish, A., Ahmad, A., Hutchison, A., Abdagic, A., Carl, A., Shen, A., Brock, A., Coenen, A., Laforge, A., Paterson, A., Bastian, B., Piot, B., Wu, B., Royal, B., Chen, C., Kumar, C., Perry, C., Welty, C., Choquette-Choo, C. A., Sinopalnikov, D., Weinberger, D., Vijaykumar, D., Rogozińska, D., Herbison, D., Bandy, E., Wang, E., Noland, E., Moreira, E., Senter, E., Eltyshev, E., Visin, F., Rasskin, G., Wei, G., Cameron, G., Martins, G., Hashemi, H., Klimczak-Plucińska, H., Batra, H., Dhand, H., Nardini, I., Mein, J., Zhou, J., Svensson, J., Stanway, J., Chan, J., Zhou, J. P., Carrasqueira, J., Iljazi, J., Becker, J., Fernandez, J., van Amersfoort, J., Gordon, J., Lipschultz, J., Newlan, J., yeong Ji, J., Mohamed, K., Badola, K., Black, K., Millican, K., McDonell, K., Nguyen, K., Sodhia, K., Greene, K., Sjoesund, L. L., Usui, L., Sifre, L., Heuermann, L., Lago, L., McNealus, L., Soares, L. B., Kilpatrick, L., Dixon, L., Martins, L., Reid, M., Singh, M., Iverson, M., Görner, M., Velloso, M., Wirth, M., Davidow, M., Miller, M., Rahtz, M., Watson, M., Risdal, M., Kazemi, M., Moynihan, M., Zhang, M., Kahng, M., Park, M., Rahman, M., Khatwani, M., Dao, N., Bardoliwalla, N., Devanathan, N., Dumai, N., Chauhan, N., Wahltinez, O., Botarda, P., Barnes, P., Barham, P., Michel, P., Jin, P., Georgiev, P., Culliton, P., Kuppala, P., Comanescu,

R., Merhej, R., Jana, R., Rokni, R. A., Agarwal, R., Mullins, R., Saadat, S., Carthy, S. M., Cogan, S., Perrin, S., Arnold, S. M. R., Krause, S., Dai, S., Garg, S., Sheth, S., Ronstrom, S., Chan, S., Jordan, T., Yu, T., Eccles, T., Hennigan, T., Kocisky, T., Doshi, T., Jain, V., Yadav, V., Meshram, V., Dharmadhikari, V., Barkley, W., Wei, W., Ye, W., Han, W., Kwon, W., Xu, X., Shen, Z., Gong, Z., Wei, Z., Cotruta, V., Kirk, P., Rao, A., Giang, M., Peran, L., Warkentin, T., Collins, E., Barral, J., Ghahramani, Z., Hadsell, R., Sculley, D., Banks, J., Dragan, A., Petrov, S., Vinyals, O., Dean, J., Hassabis, D., Kavukcuoglu, K., Farabet, C., Buchatskaya, E., Borgeaud, S., Fiedel, N., Joulin, A., Kenealy, K., Dadashi, R., and Andreev, A. Gemma 2: Improving open language models at a practical size, 2024. URL https://arxiv.org/abs/2408.00118.

Team, G., Kamath, A., Ferret, J., Pathak, S., Vieillard, N., Merhej, R., Perrin, S., Matejovicova, T., Ramé, A., Rivière, M., Rouillard, L., Mesnard, T., Cideron, G., bastien Grill, J., Ramos, S., Yvinec, E., Casbon, M., Pot, E., Penchev, I., Liu, G., Visin, F., Kenealy, K., Beyer, L., Zhai, X., Tsitsulin, A., Busa-Fekete, R., Feng, A., Sachdeva, N., Coleman, B., Gao, Y., Mustafa, B., Barr, I., Parisotto, E., Tian, D., Eyal, M., Cherry, C., Peter, J.-T., Sinopalnikov, D., Bhupatiraju, S., Agarwal, R., Kazemi, M., Malkin, D., Kumar, R., Vilar, D., Brusilovsky, I., Luo, J., Steiner, A., Friesen, A., Sharma, A., Sharma, A., Gilady, A. M., Goedeckemeyer, A., Saade, A., Feng, A., Kolesnikov, A., Bendebury, A., Abdagic, A., Vadi, A., György, A., Pinto, A. S., Das, A., Bapna, A., Miech, A., Yang, A., Paterson, A., Shenoy, A., Chakrabarti, A., Piot, B., Wu, B., Shahriari, B., Petrini, B., Chen, C., Lan, C. L., Choquette-Choo, C. A., Carey, C., Brick, C., Deutsch, D., Eisenbud, D., Cattle, D., Cheng, D., Paparas, D., Sreepathihalli, D. S., Reid, D., Tran, D., Zelle, D., Noland, E., Huizenga, E., Kharitonov, E., Liu, F., Amirkhanyan, G., Cameron, G., Hashemi, H., Klimczak-Plucińska, H., Singh, H., Mehta, H., Lehri, H. T., Hazimeh, H., Ballantyne, I., Szpektor, I., Nardini, I., Pouget-Abadie, J., Chan, J., Stanton, J., Wieting, J., Lai, J., Orbay, J., Fernandez, J., Newlan, J., yeong Ji, J., Singh, J., Black, K., Yu, K., Hui, K., Vodrahalli, K., Greff, K., Qiu, L., Valentine, M., Coelho, M., Ritter, M., Hoffman, M., Watson, M., Chaturvedi, M., Moynihan, M., Ma, M., Babar, N., Noy, N., Byrd, N., Roy, N., Momchev, N., Chauhan, N., Sachdeva, N., Bunyan, O., Botarda, P., Caron, P., Rubenstein, P. K., Culliton, P., Schmid, P., Sessa, P. G., Xu, P., Stanczyk, P., Tafti, P., Shivanna, R., Wu, R., Pan, R., Rokni, R., Willoughby, R., Vallu, R., Mullins, R., Jerome, S., Smoot, S., Girgin, S., Iqbal, S., Reddy, S., Sheth, S., Põder, S., Bhatnagar, S., Panyam, S. R., Eiger, S., Zhang, S., Liu, T., Yacovone, T., Liechty, T., Kalra, U., Evci, U., Misra, V., Roseberry, V., Feinberg, V., Kolesnikov, V., Han, W., Kwon, W., Chen, X., Chow, Y., Zhu, Y., Wei, Z., Egyed, Z., Cotruta, V., Giang, M., Kirk, P., Rao, A., Black, K., Babar, N., Lo, J., Moreira, E., Martins, L. G., Sanseviero, O., Gonzalez, L., Gleicher, Z., Warkentin, T., Mirrokni, V., Senter, E., Collins, E., Barral, J., Ghahramani, Z., Hadsell, R., Matias, Y., Sculley, D., Petrov, S., Fiedel, N., Shazeer, N., Vinyals, O., Dean, J., Hassabis, D., Kavukcuoglu, K., Farabet, C., Buchatskaya, E., Alayrac, J.-B., Anil, R., Dmitry, Lepikhin, Borgeaud, S., Bachem, O., Joulin, A., Andreev, A., Hardin, C., Dadashi, R., and Hussenot, L. Gemma 3 technical report, 2025. URL https://arxiv.org/abs/2503.19786.

Zhang, P., Liu, Z., Xiao, S., Shao, N., Ye, Q., and Dou, Z. Long context compression with activation beacon. In *The Thirteenth International Conference on Learning Representations*, 2025. URL https://openreview.net/forum?id=1eQT9OzfNQ.

A. Hyperparameters

We used 10 samples from PG19 for most experiments. Across all settings, the base language model was frozen; only the compression-token parameters were optimized (either the compression embeddings, or the low-dimensional embedding together with its projection). We optimized with AdamW. For each model, we swept the learning rate over $\{0.01, 0.1, 0.5, 1.0\}$; the resulting metrics are summarized in Table 12. For the low-dimensional embedding and projection variant, we used a learning rate of 0.01. We ran up to 10,000 optimization steps per sample in all experiments; for progressive runs, we additionally capped the optimization of each newly added token at 1,000 steps. We used 100 warmup steps for all models.

B. Compute

Each experiment ran on a single NVIDIA A100 80GB GPU, with up to 16 GPUs used in parallel across experiments. Training time depends on the sequence length and loss type, ranging from a few minutes (short sequences, small models) to approximately 24 hours (long sequences, large models). The total compute used for this research (including debugging and unsuccessful experiments) is approximately 2800 GPU-hours.

C. Full cramming vs progressive.

In Table 11 we present results from Section 5 for full and compressive cramming comparison for different model sizes of Llama and Pythia families.

D. Progressive across models scales

Table 13 shows that larger models tend to use more trajectory directions (higher PCA 99%).

E. Prefix tuning

Prefix tuning introduces a separate trainable prefix embedding at each transformer layer, avoiding the per-layer forward-pass bottleneck of soft-prompt setups such as full or progressive cramming. The trade-off is parameter cost: it requires L times more trainable embeddings, where L is the number of hidden layers in the LLM.

To separate "cramming" from general soft-prompt steering, we evaluate prefix tuning baselines. Table 14 reports the same attention-mass metric for prefix tokens. Compared to cramming, prefix tuning typically attracts less attention mass than BOS and shows weaker or more variable correlation patterns, indicating that attention hijacking is not an inevitable consequence of prepending learned embeddings but depends on the optimization objective.

Table 11. Full vs. progressive cramming on PG19. "Full" cramming uses a fixed token budget and may stop below perfect reconstruction (Kuratov et al., 2025); progressive cramming fixes reconstruction at 100% and reports the achieved token count.

Type	Tokens	Info Gain	Accuracy
Llama-3.2-1B			
Full	512	1965 ± 244	0.998 ± 0
Progr.	402 ± 85	1500 ± 282	1.0
Llama-3.2-3B			
Full	1024	3572 ± 549	0.996 ± 0.007
Progr.	902 ± 207	3074 ± 527	1.0
Llama-3.1-8B			
Full	1568	5002 ± 850	0.997 ± 0.003
Progr.	1064 ± 394	3028 ± 1321	1.0
Pythia160m			
Full	32	105 ± 20	0.684 ± 0.175
Progr.	11 ± 2	19 ± 26	1.0
Pythia410m			
Full	128	365 ± 40	0.891 ± 0.062
Progr.	102 ± 41	323 ± 105	1.0
Pythia1.4b			
Full	256	802 ± 113	0.985 ± 0.031
Progr.	544 ± 57	1694 ± 125	1.0

We also evaluated compression limits under prefix tuning. Table 15 shows that, for Llama 3 models at 3B and 8B, we cannot reach the cramming limits; however, in the prefix-tuning setup these models can still compress up to 8k tokens with perfect reconstruction accuracy, while 16k tokens results in out-of-memory (OOM) failures. For the 1B Llama 3 model, the limit is about 4096 tokens, which is roughly $10\times$ higher than with progressive cramming. This gain comes at a higher parameter cost: prefix tuning requires L-times more trainable embeddings (one per layer), i.e., $16\times$ more parameters for a 16-layer model. For Pythia, the prefix-tuning setup increases the compression limit by about $20\times$ for Pythia-410M and by about $8\times$ for Pythia-1.4B relative to progressive cramming.

F. Dataset Modifications

To probe sensitivity to surface form, we create controlled variants of PG19 samples and compare their progressive trajectories (Figure 6). Each variant shares an identical 64-token prefix and modifies the suffix in one of three ways: random word permutation, greedy decoding from Llama-3.1-8B ("Sampled"), or lowercasing. Trajectories coincide during the shared prefix and diverge immediately after the edit point, indicating that the optimization path is highly sensitive to token-level details. Notably, lowercasing (which should preserve high-level semantics) induces a markedly different trajectory, suggesting that progressive cramming is driven by tokenization- and distribution-level properties rather than semantic equivalence.

Table 12. Results of the learning-rate sweep across models and settings. "Compressed Tokens" is the achieved prefix length n, "Trajectory Length" is L_{traj}, and "PCA 99%" is the number of principal components explaining 99% of trajectory variance.

Model	Compressed Tokens	Information Gain	Trajectory Length	PCA 99%
Llama-3.1-8B	1298.4 ± 537.5	3760 ± 1418	738 ± 265	40.8 ± 13.16
Llama-3.1-8B lr=0.1	1063.5 ± 394.4	3028 ± 1321	4861 ± 1033	74.4 ± 7.94
Llama-3.1-8B lr=0.5	934.4 ± 123.2	2758 ± 486	15424 ± 1437	138 ± 11.78
Llama-3.1-8B lr=1.0	1286.4 ± 524.8	3381 ± 1521	29061 ± 4605	186.6 ± 13.59
pythia-1.4b	188.2 ± 27.8	581 ± 101	83 ± 14	15.7 ± 1.85
pythia-1.4b lr=0.1	274.3 ± 56.2	846 ± 106	956 ± 123	23.2 ± 1.99
pythia-1.4b lr=0.5	543.8 ± 56.8	1694 ± 125	6730 ± 390	49 ± 3.32
pythia-1.4b lr=1.0	558.2 ± 74.1	1737 ± 158	11381 ± 993	63.3 ± 6.21
SmolLM2-1.7B lr=0.1	370.2 ± 113.1	1119 ± 350	1027 ± 167	29.2 ± 2.93
SmolLM2-1.7B lr=0.5	540.5 ± 139.4	1450 ± 350	5225 ± 827	66.8 ± 9.14
SmolLM2-1.7B lr=1.0	502.5 ± 180.6	1428 ± 473	9442 ± 1810	83 ± 16.34
gemma-3-4b-pt lr=0.1	286.5 ± 164.6	949 ± 466	1044 ± 328	31.8 ± 9.21
gemma-3-4b-pt lr=0.5	498.6 ± 86.5	1437 ± 308	6055 ± 925	71.6 ± 11.34
gemma-3-4b-pt lr=1.0	558 ± 86.1	1258 ± 511	12173 ± 1004	101.6 ± 10.66

Table 13. Progressive cramming trajectory statistics on PG19. "Compressed Tokens" is the achieved prefix length n, "Trajectory Length" is L_{traj}, and "PCA 99%" is the number of principal components explaining 99% of trajectory variance.

Model	Compressed Tokens	Information Gain	Trajectory Length	PCA 99%
Llama-3.2-1B lr=0.1	402.2 ± 84.8	1500 ± 282	1736 ± 206	44.1 ± 5.73
Llama-3.2-3B lr=0.1	902.2 ± 206.8	3074 ± 527	4232 ± 913	61.5 ± 4.54
Llama-3.1-8B lr=0.1	1063.5 ± 394.4	3028 ± 1321	4861 ± 1033	74.4 ± 7.94
pythia-160m lr=0.5	10.7 ± 2	19 ± 26	652 ± 196	5.1 ± 1.64
pythia-410m lr=0.5	102.1 ± 40.8	323 ± 105	3613 ± 841	37.5 ± 11.16
pythia-1.4b lr=0.5	543.8 ± 56.8	1694 ± 125	6730 ± 390	49 ± 3.32
SmolLM2-135M lr=0.1	38.5 ± 14.1	168 ± 66	178 ± 40	12.2 ± 2.96
SmolLM2-360M lr=0.1	61 ± 24.6	266 ± 122	234 ± 111	13.4 ± 3.95
SmolLM2-1.7B lr=0.1	370.2 ± 113.1	1119 ± 350	1027 ± 167	29.2 ± 2.93
gemma-3-270m lr=0.1	71.2 ± 14.2	392 ± 71	259 ± 32	18.5 ± 2.54
gemma-3-1b-pt lr=0.1	74.8 ± 34.3	338 ± 104	386 ± 89	18.9 ± 6.59
gemma-3-4b-pt lr=0.1	286.5 ± 164.6	949 ± 466	1044 ± 328	31.8 ± 9.21

G. Full activation alignment experiments

Table 16 reports the full sweep over activation alignment strength α, number of aligned layers L, and projection dimension, together with the resulting progressive cramming metrics.

H. Full Low dimensional experiments

Table 17 ablates projection dimension across model families. Three takeaways stand out:

- For Llama-3.1-8B and SmolLM2-1.7B, projection can substantially increase achievable token counts and information gain compared to the unconstrained baseline.
- Increasing projection dimension does not reliably increase PCA 99% (and can even decrease it), but it often increases the measured trajectory length, suggesting longer optimization paths within a similarly low-dimensional subspace.
- For Pythia-1.4B, small projection dimensions strongly reduce PCA 99% and trajectory length, but also reduce capacity; adding a larger projection partially recovers information gain.

I. Analysis of the Recostruction Accuracy

As noted in Section 3.2, a 99% token-level accuracy threshold is insufficient to guarantee correct reconstruction when evaluation is performed without *teacher forcing* (i.e., when generated tokens are fed back into the prefix rather than replacing them with the ground-truth tokens). Table 18 shows that under non-progressive training, the remaining $\sim$1% error is concentrated in the first few positions (indices 0-2); these early mistakes then compound, leading to divergent generations.

Table 14. Prefix-tuning attention mass, computed with the same metric as Table 3 but for learned prefix tokens instead of the compression embedding.

Model	Compression Emb. (%)	BOS Token Original (%)	Diff (%)	Correlation
SmolLM2-135M	26.53 ± 2.78	41.74 ± 6.35	-15.21 ± 7.13	0.8691
SmolLM2-360M	28.07 ± 4.8	49.81 ± 7.67	-21.74 ± 8.13	0.6066
SmolLM2-1.7B	36.43 ± 4.58	44.03 ± 6.73	-7.61 ± 8.38	0.9102
Llama-3.2-3B	23.24 ± 4.55	67.75 ± 1.18	-44.51 ± 5.61	0.5641
Qwen3-4B	1.95 ± 0.22	50.84 ± 1.54	-48.89 ± 1.46	0.0771

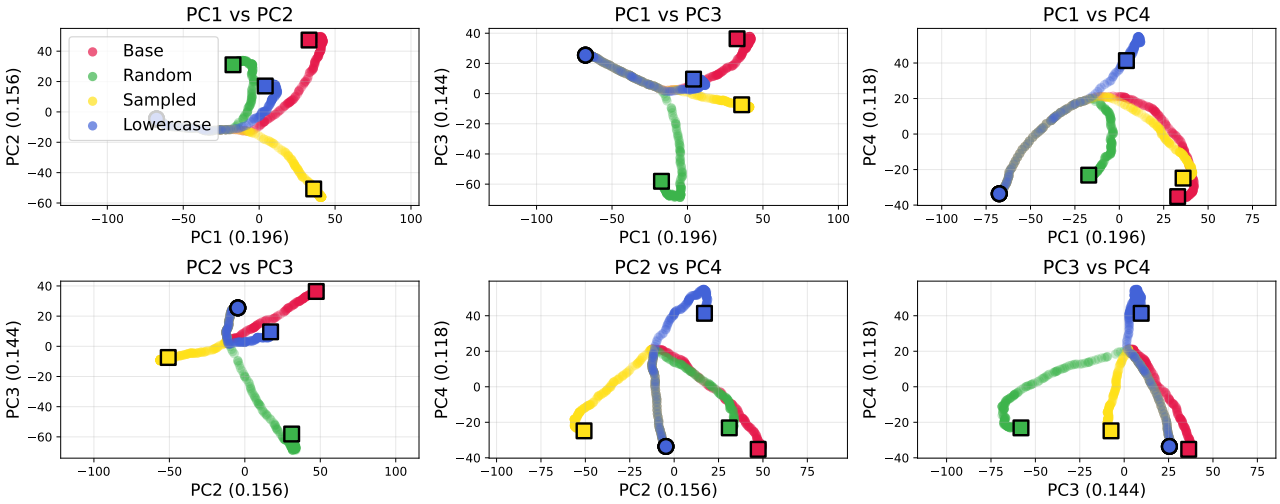

Figure 6. Progressive trajectories projected onto PCA components (Llama-3.1-8B, one PG19 sample). All variants share the same 64-token prefix; the suffix is either original (Base), randomly permuted (Random), greedily sampled continuation (Sampled), or lowercased (Lowercased). Circles mark the start and squares mark the end. Axis titles report the signed explained-variance fraction.

Table 15. Prefix-tuning reconstruction accuracy compared to progressive cramming. "Tokens" denotes the number of learned prefix tokens (or reconstructed tokens for progressive cramming).

Experiment	Type	Tokens	Accuracy
Llama-3.2-1B	Progr.	402 ± 85	1.0
Llama-3.2-1B	Full PrefixT.	4096	1 ± 0
Llama-3.2-1B	Full PrefixT.	8192	0.977 ± 0.032
Llama-3.2-1B	Full PrefixT.	16384	0.767 ± 0.105
Llama-3.2-3B	Progr.	902 ± 207	1.0
Llama-3.2-3B	Full PrefixT.	8192	1 ± 0
Llama-3.1-8B	Progr.	1064 ± 394	1.0
Llama-3.1-8B	Full PrefixT.	8192	1 ± 0
Pythia160m	Progr.	11 ± 2	1.0
Pythia160m	Full PrefixT.	1024	0.488 ± 0.06
Pythia160m	Full PrefixT.	2048	0.394 ± 0.034
Pythia160m	Full PrefixT.	4096	0.263 ± 0.025
Pythia160m	Full PrefixT.	8192	0.168 ± 0.018
Pythia160m	Full PrefixT.	16384	0.139 ± 0.026
Pythia410m	Progr.	102 ± 41	1.0
Pythia410m	Full PrefixT.	2048	1 ± 0
Pythia410m	Full PrefixT.	4096	0.964 ± 0.071
Pythia410m	Full PrefixT.	8192	0.588 ± 0.106
Pythia410m	Full PrefixT.	16384	0.418 ± 0.158
Pythia1.4b	Progr.	544 ± 57	1.0
Pythia1.4b	Full PrefixT.	4096	1 ± 0
Pythia1.4b	Full PrefixT.	8192	0.979 ± 0.015
Pythia1.4b	Full PrefixT.	16384	0.663 ± 0.12

Table 16. Full sweep of activation alignment and low-dimensional projection hyperparameters. "α" is alignment strength, "L" is the number of aligned layers, and "dim" is projection dimension.

Model	Compressed Tokens	Information Gain	Trajectory Length	PCA 99%
Llama-3.1-8B $\alpha = 1.0$ $L = 2$	1532.3 ± 280	4694 ± 341	812 ± 167	45.8 ± 5.29
Llama-3.1-8B $\alpha = 1.0$ $L = 4$	1593.6 ± 306.6	4961 ± 603	958 ± 207	41.9 ± 3.01
Llama-3.1-8B $\alpha = 1.0$ $L = 8$	1633.3 ± 275.4	5130 ± 247	1146 ± 137	40 ± 1.26
Llama-3.1-8B $\alpha = 1.0$ $L = 16$	1487.3 ± 255.4	4686 ± 276	1146 ± 112	33.6 ± 2.2
Llama-3.1-8B $\alpha = 1.0$ $L = 24$	664.8 ± 99.2	2095 ± 247	570 ± 87	20.5 ± 1.75
Llama-3.1-8B $\alpha = 1.0$ $L = 32$	192.7 ± 50.6	614 ± 169	150 ± 31	12.6 ± 1.69
Llama-3.1-8B dim=32 $\alpha = 1.0$ $L = 8$	1804.8 ± 297.8	5103 ± 748	13584 ± 3823	34.7 ± 5.37
Llama-3.1-8B dim=64 $\alpha = 1.0$ $L = 8$	1853.3 ± 256.2	5512 ± 456	14367 ± 3209	30.8 ± 5.27
Llama-3.1-8B dim=128 $\alpha = 1.0$ $L = 8$	1858.9 ± 283.6	5145 ± 1695	18967 ± 3766	33.5 ± 2.84
Llama-3.1-8B dim=256 $\alpha = 1.0$ $L = 8$	1810.6 ± 236	5105 ± 1693	23576 ± 5844	34.1 ± 5.5
pythia-1.4b $\alpha = 1.0$ $L = 4$	182.7 ± 31.1	553 ± 63	83 ± 9	14.6 ± 1.96
pythia-1.4b $\alpha = 1.0$ $L = 8$	178.7 ± 33.1	524 ± 92	80 ± 13	14.3 ± 1.79
pythia-1.4b $\alpha = 1.0$ $L = 16$	155.5 ± 35.5	427 ± 66	69 ± 16	13.5 ± 1.5
pythia-1.4b $\alpha = 1.0$ $L = 20$	116.6 ± 26.1	307 ± 35	58 ± 10	11.9 ± 0.83
pythia-1.4b dim=32 $\alpha = 1.0$ $L = 8$	423.1 ± 104.4	1317 ± 221	2498 ± 1438	15.5 ± 4.57
pythia-1.4b dim=64 $\alpha = 1.0$ $L = 8$	447.8 ± 81.9	1398 ± 186	3309 ± 2569	16.9 ± 4.01
pythia-1.4b dim=128 $\alpha = 1.0$ $L = 8$	468.3 ± 68.6	1448 ± 106	4699 ± 2414	20 ± 4.96
pythia-1.4b dim=256 $\alpha = 1.0$ $L = 8$	498.2 ± 92.7	1555 ± 146	5396 ± 3309	19.6 ± 5.89
SmolLM2-1.7B $\alpha = 1.0$ $L = 8$	148.1 ± 56.7	432 ± 157	75 ± 18	11.9 ± 1.97
SmolLM2-1.7B $\alpha = 1.0$ $L = 4$	186.9 ± 47.3	530 ± 104	85 ± 8	13.3 ± 1.73
SmolLM2-1.7B $\alpha = 1.0$ $L = 16$	126.5 ± 63.3	366 ± 155	66 ± 22	11.3 ± 2.19
SmolLM2-1.7B $\alpha = 1.0$ $L = 20$	96.1 ± 56.2	254 ± 114	50 ± 20	8.8 ± 3.54
SmolLM2-1.7B dim=32 $\alpha = 1.0$ $L = 8$	292.7 ± 113.3	851 ± 327	1144 ± 705	11.1 ± 3.99
SmolLM2-1.7B dim=64 $\alpha = 1.0$ $L = 8$	321.4 ± 148.3	914 ± 305	1668 ± 2014	10.4 ± 2.29
SmolLM2-1.7B dim=128 $\alpha = 1.0$ $L = 8$	363.2 ± 67.2	1065 ± 213	2236 ± 1198	10.4 ± 1.5
SmolLM2-1.7B dim=256 $\alpha = 1.0$ $L = 8$	468 ± 243.7	1350 ± 621	6544 ± 9765	11.6 ± 3.61
gemma-3-4b-pt $\alpha = 1.0$ $L = 4$	60.3 ± 52.7	224 ± 163	60 ± 20	9.6 ± 3.01
gemma-3-4b-pt $\alpha = 1.0$ $L = 8$	75.1 ± 84.9	268 ± 256	65 ± 44	9.3 ± 3.93
gemma-3-4b-pt $\alpha = 1.0$ $L = 16$	55.8 ± 42.8	217 ± 156	57 ± 19	8.4 ± 2.65
gemma-3-4b-pt $\alpha = 1.0$ $L = 20$	48.3 ± 31.4	191 ± 104	55 ± 16	8.3 ± 2.45
gemma-3-4b-pt dim=32 $\alpha = 1.0$ $L = 8$	121.3 ± 127.9	393 ± 317	429 ± 277	9.3 ± 3.93
gemma-3-4b-pt dim=64 $\alpha = 1.0$ $L = 8$	327.5 ± 224.5	1082 ± 708	807 ± 502	13.4 ± 3.58
gemma-3-4b-pt dim=128 $\alpha = 1.0$ $L = 8$	361.3 ± 214.4	1209 ± 672	1511 ± 801	14.5 ± 2.06
gemma-3-4b-pt dim=256 $\alpha = 1.0$ $L = 8$	217.1 ± 146.2	719 ± 425	1091 ± 557	13.3 ± 2.69
Qwen3-4B $\alpha = 1.0$ $L = 4$	166.9 ± 56.1	699 ± 193	104 ± 18	13.8 ± 1.54
Qwen3-4B $\alpha = 1.0$ $L = 8$	197.1 ± 74	792 ± 243	118 ± 25	15.1 ± 2.12
Qwen3-4B $\alpha = 1.0$ $L = 16$	170.9 ± 71.3	711 ± 265	109 ± 28	13.9 ± 2.47
Qwen3-4B $\alpha = 1.0$ $L = 20$	168.9 ± 39.6	695 ± 132	106 ± 12	13.9 ± 1.37
Qwen3-4B dim=32 $\alpha = 1.0$ $L = 8$	774.9 ± 156.4	3063 ± 556	4243 ± 1543	25.9 ± 4.48
Qwen3-4B dim=64 $\alpha = 1.0$ $L = 8$	811.8 ± 189.1	3181 ± 567	7120 ± 2953	30.7 ± 4.17
Qwen3-4B dim=128 $\alpha = 1.0$ $L = 8$	870.8 ± 160.5	3400 ± 300	9040 ± 3358	31 ± 5.42
Qwen3-4B dim=256 $\alpha = 1.0$ $L = 8$	895.7 ± 219.4	3470 ± 476	9813 ± 3019	32.5 ± 4.25

Table 17. Effect of low-dimensional projection size on progressive cramming. "dim" in the model column denotes the projection dimension k (Section 4.3).

Model	Compressed Tokens	Information Gain	Trajectory Length	PCA 99%
Llama-3.1-8B lr=0.1	1063.5 ± 394.4	3028 ± 1321	4861 ± 1033	74.4 ± 7.94
Llama-3.1-8B dim=32	1745 ± 306	5312 ± 330	10250 ± 3256	35.7 ± 5.1
Llama-3.1-8B dim=64	1834.7 ± 251	5694 ± 333	9977 ± 2641	32.5 ± 4.3
Llama-3.1-8B dim=128	1834.6 ± 305.6	5538 ± 537	18713 ± 10220	32.8 ± 4.6
Llama-3.1-8B dim=256	1730.8 ± 384.3	[4870] ± 1680	21448 ± 5583	29.4 ± 3.29
Llama-3.1-8B dim=512	1652.9 ± 404.5	5072 ± 799	25852 ± 7528	36.2 ± 5.88
pythia-1.4b lr=0.5	543.8 ± 56.8	1694 ± 125	6730 ± 390	49 ± 3.32
pythia-1.4b dim=32	358.2 ± 80.6	1137 ± 241	1551 ± 954	15.1 ± 3.01
pythia-1.4b dim=64	392.8 ± 96.6	1240 ± 286	1920 ± 1191	16.7 ± 3.69
pythia-1.4b dim=128	373 ± 81.8	1168 ± 240	1915 ± 1133	15.8 ± 3.22
pythia-1.4b dim=256	375.5 ± 100.7	1189 ± 314	2135 ± 1312	16.5 ± 2.16
pythia-1.4b dim=512	415.5 ± 76.3	1333 ± 200	2863 ± 1433	18 ± 2.49
SmolLM2-1.7B lr=0.1	370.2 ± 113.1	1119 ± 350	1027 ± 167	29.2 ± 2.93
SmolLM2-1.7B dim=32	335.8 ± 78.7	1007 ± 284	543 ± 218	14.2 ± 1.99
SmolLM2-1.7B dim=64	403.2 ± 77.7	1195 ± 177	778 ± 167	13.8 ± 1.94
SmolLM2-1.7B dim=128	431.9 ± 115.6	1252 ± 288	1236 ± 653	13.9 ± 2.51
SmolLM2-1.7B dim=256	487.7 ± 152.7	1449 ± 402	2265 ± 1405	16 ± 4.47
SmolLM2-1.7B dim=512	557.1 ± 158.8	1615 ± 350	2590 ± 1163	17.7 ± 3.95
gemma-3-4b-pt lr=0.1	286.5 ± 164.6	949 ± 466	1044 ± 328	31.8 ± 9.21
gemma-3-4b-pt dim=32	233.7 ± 202.3	670 ± 428	673 ± 343	12 ± 3.1
gemma-3-4b-pt dim=64	214.9 ± 172.8	718 ± 568	700 ± 486	11.8 ± 2.71
gemma-3-4b-pt dim=128	275.5 ± 222.6	860 ± 667	1107 ± 1265	13.3 ± 2.76
gemma-3-4b-pt dim=256	252.7 ± 189.9	829 ± 635	1372 ± 886	13.1 ± 4.83
gemma-3-4b-pt dim=512	217.4 ± 169.1	639 ± 439	1297 ± 975	12.3 ± 5.95

Table 18. Compression reconstruction accuracy under non-ideal training convergence. We report mean teacher-forcing accuracy over all compressed samples, mean greedy decoding accuracy when generating autoregressively from the compression token, and the fraction of samples with a token mismatch at indices 0, 1, and 2. Variants include ignoring the BOS loss during training (no BOS) and upweighting the loss of the first two tokens by a factor of three (2 leading).

Model	Tokens	Setup	Final conv.	Greedy conv.	Mismatch (%)		
					@0	@1	@2
Llama-3.2-1B	512	common	0.9881	0.0073	90.0%	100.0%	0.0%
Llama-3.2-1B	512	no BOS	0.9902	0.0006	100.0%	100.0%	0.0%
Llama-3.2-1B	512	2 leading	0.9902	0.0203	0.0%	100.0%	0.0%
Llama-3.2-3B	1024	common	0.9672	0.0005	100.0%	100.0%	0.0%
Llama-3.2-3B	1024	no BOS	0.9578	0.0079	100.0%	100.0%	0.0%
Llama-3.2-3B	1024	2 leading	0.9605	0.0156	0.0%	100.0%	0.0%
Llama-3.1-8B	1568	common	0.9706	0.0030	100.0%	100.0%	0.0%
Llama-3.1-8B	1568	no BOS	0.9670	0.0078	100.0%	100.0%	0.0%
Llama-3.1-8B	1568	2 leading	0.9670	0.0165	20.0%	100.0%	0.0%